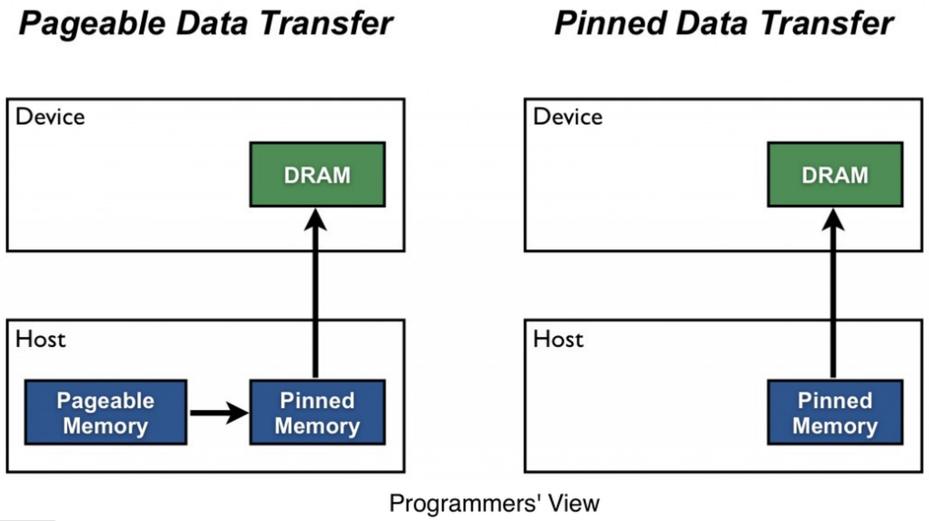
# Introduction to GPU Programming

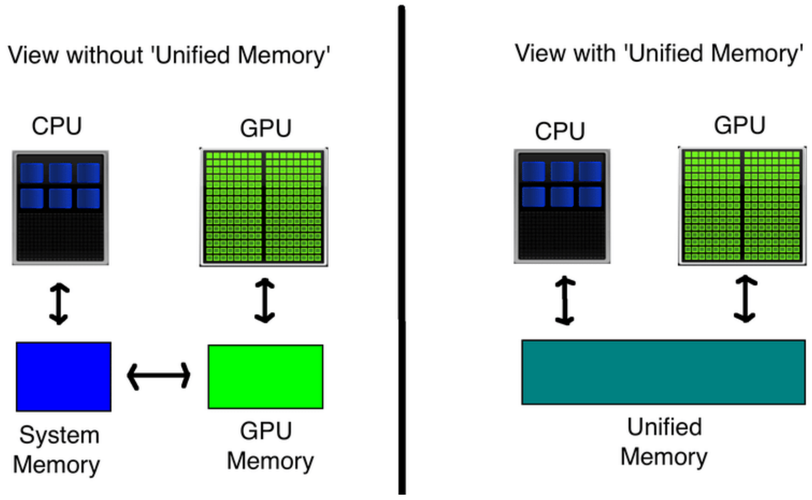
Host Memory Allocation

# CUDA Host Memory Model

- Pageable
- Pinned
- Mapped
- Unified




| Memory Type | Memory Trasfer time in ms | Kernel Execution Time in ms | Total Time in ms |
|---|---|---|---|
| Pageable | 19.43 | 0.44 | 19.87 |
| Pinned | 4.26 | 0.44 | 4.70 |
| Mapped | 0.00 | 3.72 | 3.72 |
| Unified | 1.96 | 16.13 | 18.09 |




[1]

# Host Memory Primitive Allocation Syntax

- primitives (int, float, etc.) can be allocated as simple variables:

```
int input_a, input_b;
float input_c;
```

# Host Pageable Memory Pointers Allocation Syntax

- Pointers can be used to transfer large amounts of data for later use in global memory. They can be defined as follows:

```
int input_size = 1024;
int *input_a;
input_a = (int *)malloc(input_size *sizeof(int));
```

- Arrays can be used as well, saving some of the hassle of pointer math, though this is only really small reduction in syntax, functionally the same:

```
int INPUT_SIZE = 1024;
int input_a[input_size];
```

# Host Non-Pageable Memory Pointers Allocation Syntax

- Pinned memory allocation (removes extra host-side memory transfers):

```
int input_size = 1024;
float *input_a;
cudaMallocHost((float **)&input_a, input_size*sizeof(float));
```

- Mapped memory allocation (no copies to device memory):

```
cudaHostMalloc((float **)&input_a, input_size*sizeof(float), cudaHostAllocMapped);
```

- Unified memory allocation (no need to worry about copies let system do it for you):

```
cudaMallocManaged((int **)&input_a, input_size*sizeof(float));
```

[2]

# Host Struct Allocation Syntax

- Structs add a more object-oriented approach to host memory, allowing for more complex and clearly defined data to be sent to the GPU. You can send arrays of structs for things to represent image pixels (R,G,B,A):

```
struct image_pixel{
        int red;
        int green;
        int blue;
        int alpha;
};
struct image{
        image_pixel image_pixels[1920][1080];
};

image  *example_image;
```

[2]

# Bibliography


- [1] 2020. Medium. CUDA – Memory Model. Retrieved from https://medium.com/analytics-vidhya/cuda-memory-model-823f02cef0bf.
- [2] 2021. Nvidia. CUDA Toolkit Documentation. Retrieved from https://docs.nvidia.com/cuda/cuda-runtime-api.
- [3] 2021. CPlusPlus.com. Data Structures. Retrieved from http://www.cplusplus.com/doc/tutorial/structures/.

JOHNS HOPKINS
UNIVERSITY

© The Johns Hopkins University 2021, All Rights Reserved.